

# Tutorial 3:

# Filters, Convolution &

# the Convolution Theorem

---



# Your time series tutorial team



**Noga Mudrik**



**Richard Gao**



**Gal Vishne**



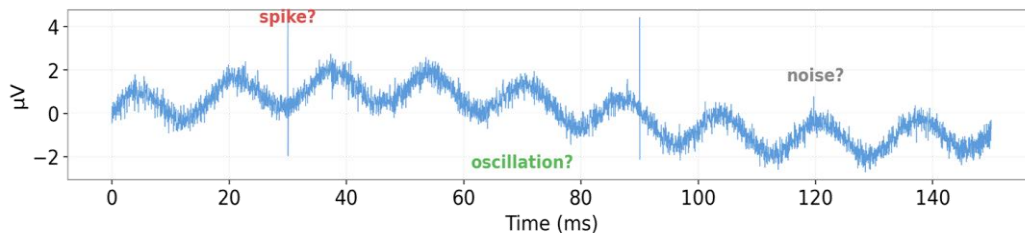
**Adam  
Manoogian**



**Ria Cheruvu**

# The Problem

- Neural recordings are a messy mix of:
  - Signal of interest (oscillations, spikes)
  - Biological noise (muscle, heartbeat)
  - Environmental noise (line noise, equipment)
- By the end of this tutorial, you will separate spikes from oscillations in real Neuropixels data.



# What You'll Learn

## 1. How convolution works as a filter

Slide a kernel over a signal → smoothed output

## 2. Why it works (the convolution theorem)

Convolution in time = multiplication in frequency

## 3. Practical pitfalls

Sharp cutoffs, phase delays, spike artifacts

## 4. Apply it

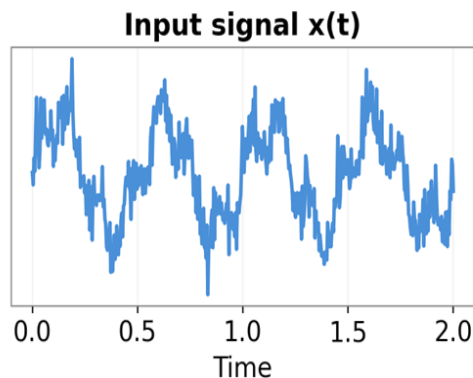
Separate spikes from LFP in real data

# Video 2:

## The Theory Behind Filtering

# Convolution Implements Filters

$$(x * h)[n] = \sum_{m=0}^{M-1} x[n-m]h[m]$$



Slide a kernel  $h(t)$  across your signal  $x(t)$ , multiply and sum at each position.  
Gaussian kernel  $\rightarrow$  smoothing. Derivative kernel  $\rightarrow$  edge detection.

# Exercise 1: Convolve Signals with a Kernel

Your turn! Convolve three different signals with a Gaussian kernel and see how each one changes.

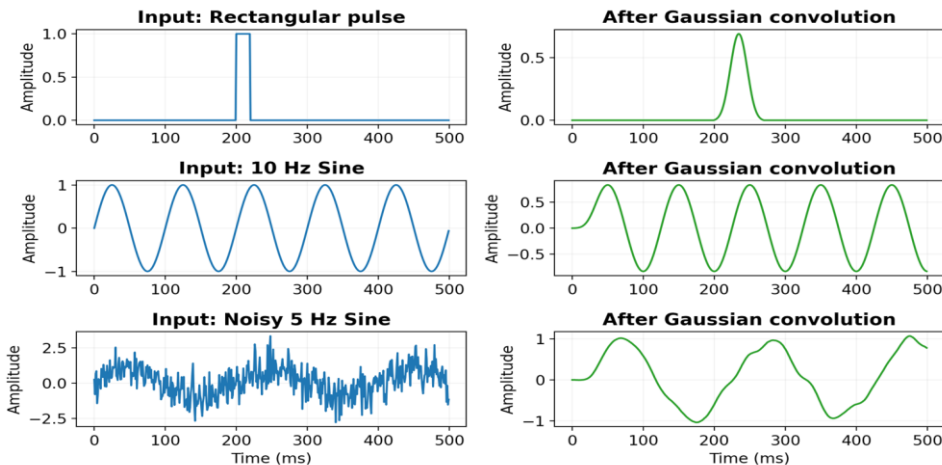
```
fs = 1000.0
n = 500
t = np.arange(n) / fs

# Three input signals
x_pulse = np.zeros(n); x_pulse[200:220] = 1.0 # rectangular pulse
x_sine = np.sin(2 * np.pi * 10 * t) # 10 Hz sine
rng = np.random.default_rng(42)
x_noisy = np.sin(2 * np.pi * 5 * t) + 0.8 * rng.standard_normal(n) # noisy sine

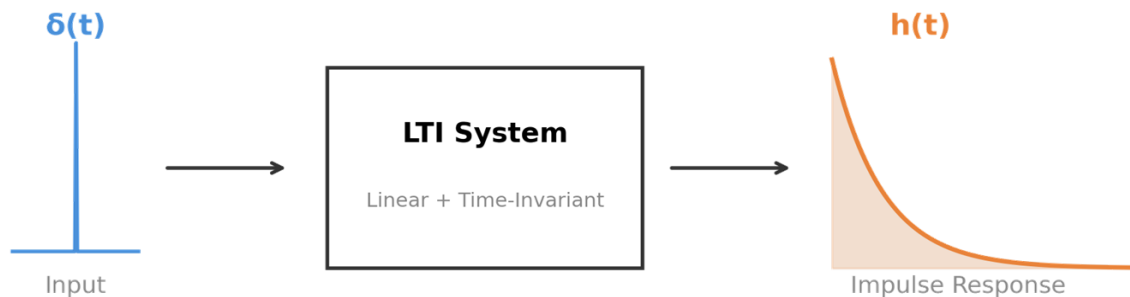
# Gaussian kernel (51 samples wide, std=10 samples)
gauss_kernel = signal.windows.gaussian(51, std=10)
gauss_kernel = gauss_kernel / gauss_kernel.sum() # normalize to sum to 1

#####
## TODO for students:
## 1. Convolve x_pulse with gauss_kernel using np.convolve(..., mode='full')
## 2. Convolve x_sine with gauss_kernel
## 3. Convolve x_noisy with gauss_kernel
## Store results in y_pulse, y_sine, y_noisy
## Then trim each to length n: y = y[:n]
#####

y_pulse = ...
y_sine = ...
y_noisy = ...
```



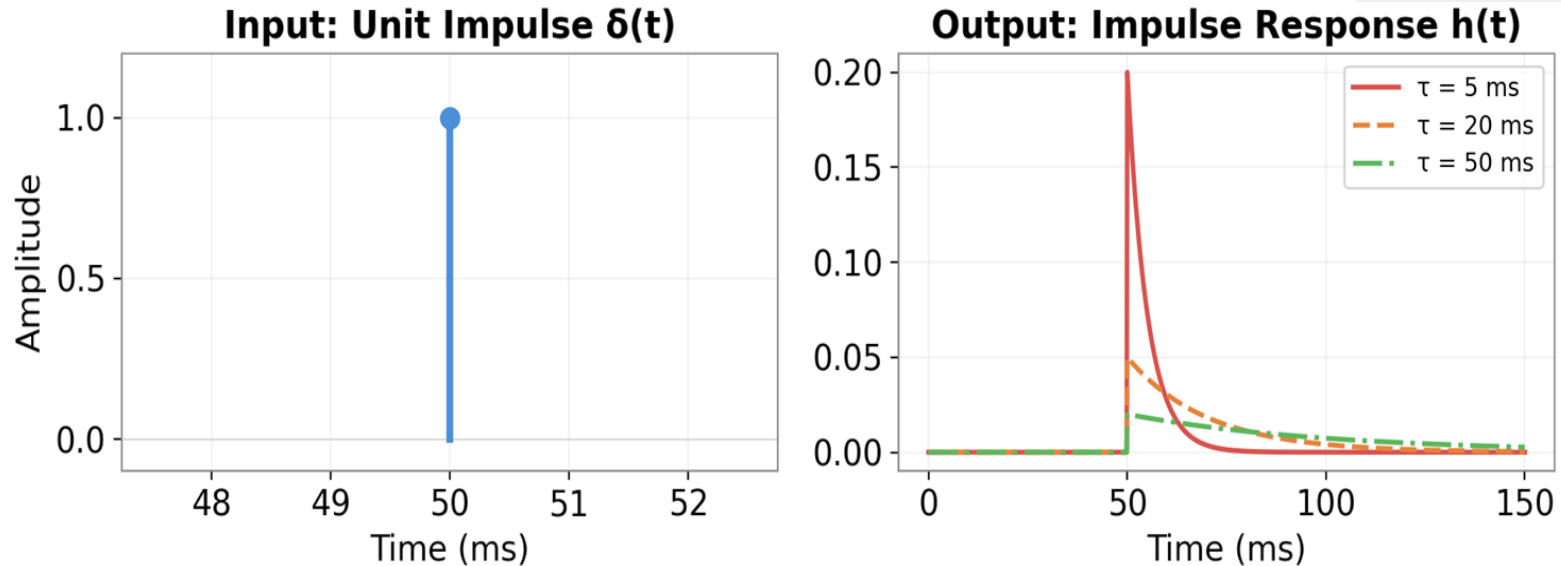
# LTI Systems & Impulse Response



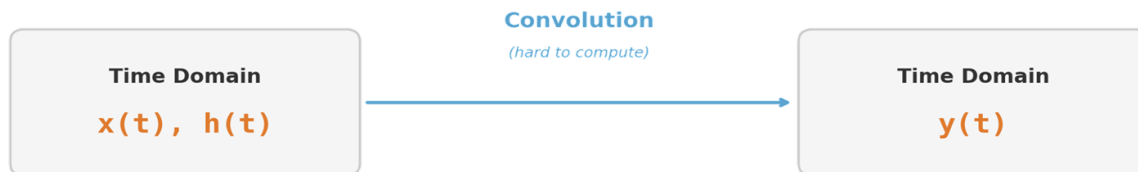
- **Linear:** scale input  $\rightarrow$  output scales the same
- **Time-Invariant:** shift input  $\rightarrow$  output shifts the same
- Feed in a spike, record what comes out  $\rightarrow$  that's  $h(t)$



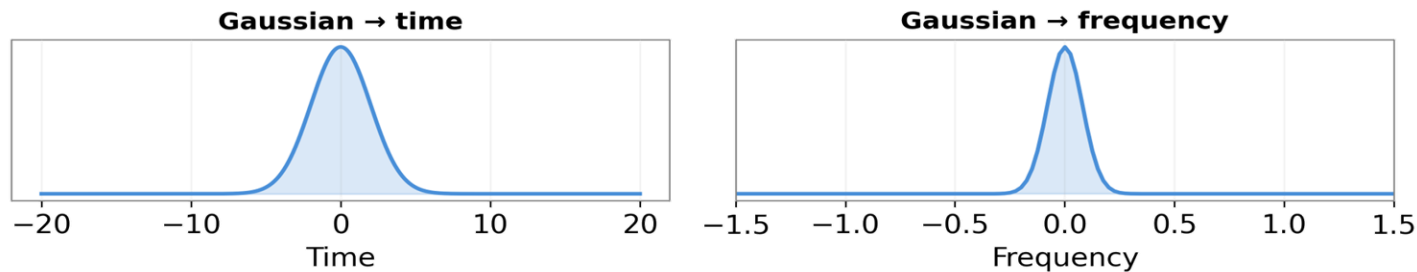
# Impulse Response: One Spike In, Shape Out



# The Convolution Theorem



# Narrow in Time $\leftrightarrow$ Wide in Frequency



# Exercise 2: Verify the Convolution Theorem

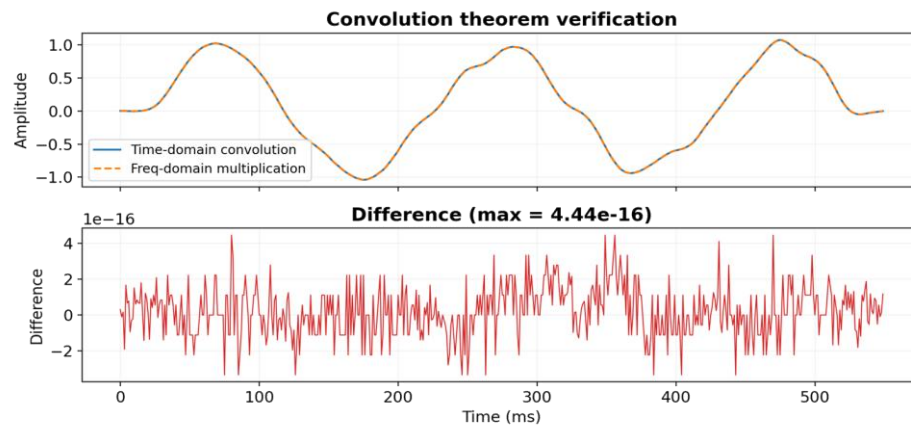
Compute the same convolution two ways: in time and in frequency, and check they match.

```
# Signal and kernel from Exercise 1
# x_noisy is already defined (noisy 5 Hz sine, length n=500)
# gauss_kernel is already defined (length 51)

conv_length = len(x_noisy) + len(gauss_kernel) - 1 # length of full convolution

#####
## TODO for students:
## 1. Compute y_time: convolve x_noisy with gauss_kernel (mode='full')
##
## 2. Compute y_freq using the frequency domain:
##   a. Zero-pad both signals to conv_length
##       x_padded = np.zeros(conv_length); x_padded[:len(x_noisy)] = x_noisy
##       h_padded = np.zeros(conv_length); h_padded[:len(gauss_kernel)] = gauss_kernel
##   b. Take FFT of both: np.fft.rfft(...)
##   c. Multiply the FFTs pointwise
##   d. Inverse FFT: np.real(np.fft.irfft(...))
##
## 3. Compare the two results
#####

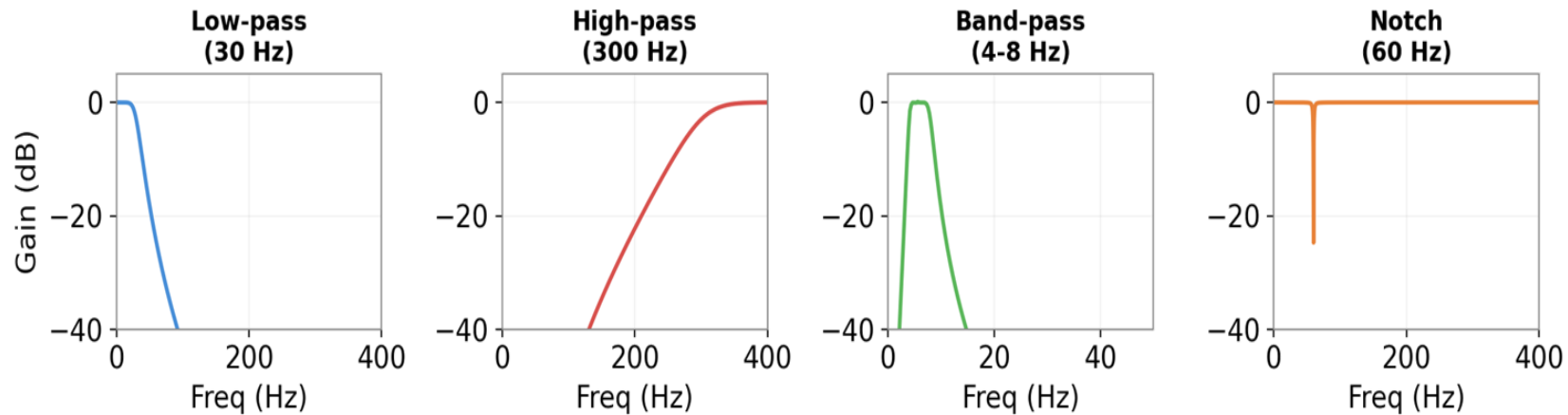
y_time = ...
y_freq = ...
```



# Video 3:

## Filter Design & Pitfalls

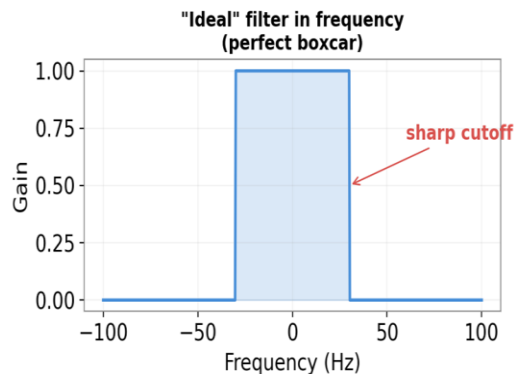
# Four Filter Types



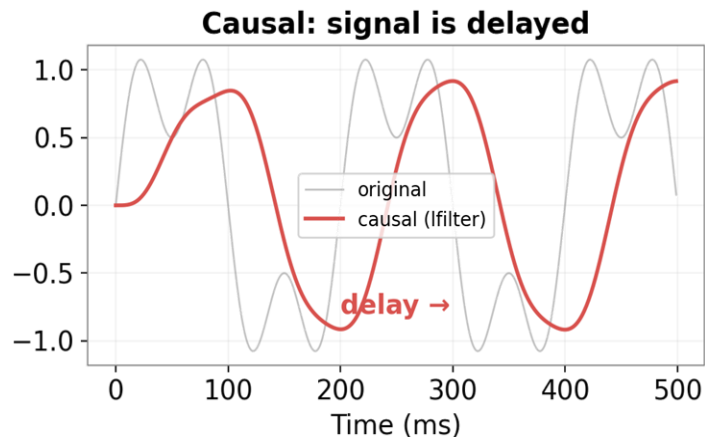
Each filter selectively passes or blocks frequency ranges. Choose based on what you want to keep.

# The Ideal Filter Trap

- Why not a perfect rectangular cutoff?
  - Boxcar in frequency = sinc in time
  - Infinite extent: impossible on finite data
  - Creates ringing artifacts at edges
- Real filters often trade sharpness for stability.



# Causal vs Zero-Phase Filtering



**Causal (lfilter):** processes left-to-right → introduces delay

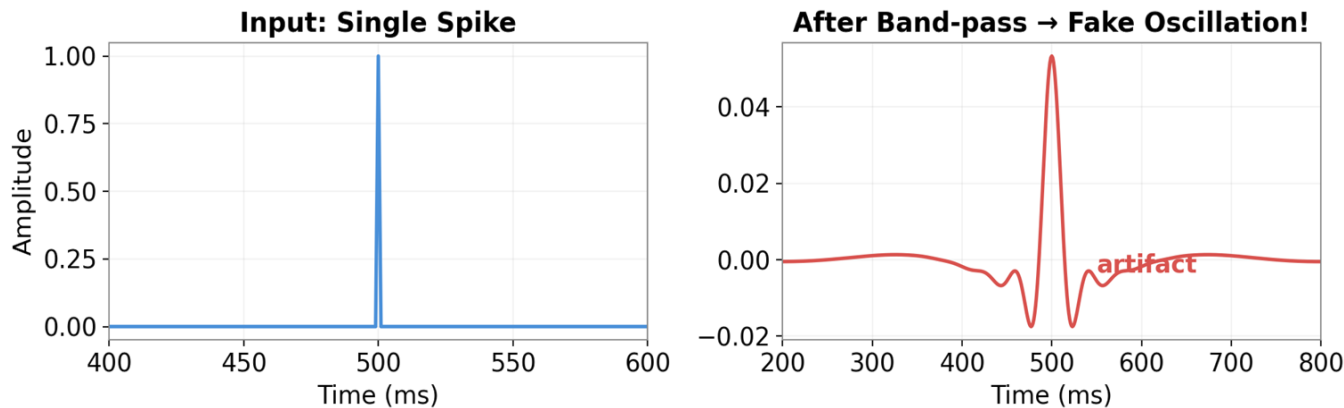
**Zero-phase (filtfilt):** forward then backward → no delay

**Rule of thumb:** for offline neuroscience analysis, use zero-phase.

In Python, that means using `filtfilt` instead of `lfilter`.



## ⚠ Pitfall: Spike → Fake Oscillation

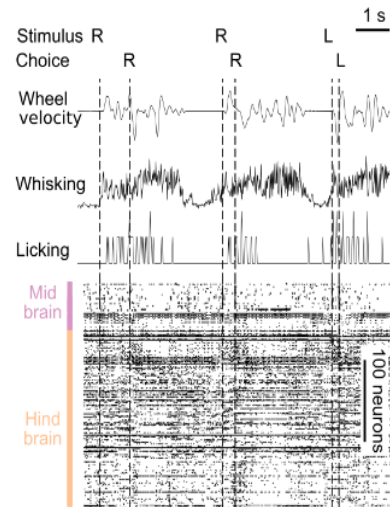
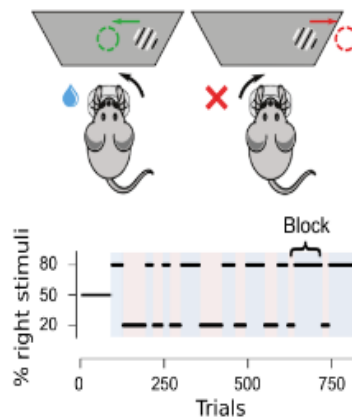


Band-pass filtering a sharp spike creates a ringing artifact that looks like a real oscillation.

**Always check if your "oscillation" survives spike removal!**

# Next Up: Real Neuropixels Data

- You will apply these filters to real neural data.
- IBL Neuropixels recordings.
- Separate LFP from spikes.



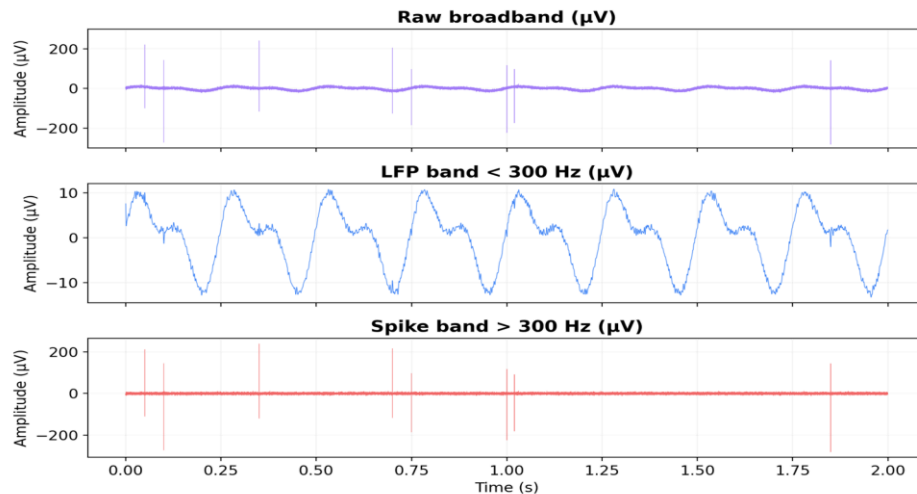
Illustrations from the IBL (2022). Data release - Brainwide map.  
Preprint. <https://doi.org/10.6084/m9.figshare.21400815.v7>

# Exercise 3: Separate LFP and Spikes

Apply low-pass and high-pass filters to real Neuropixels data to pull apart LFP and spikes.

```
#####
## TODO:
## 1. Design a low-pass Butterworth filter (cutoff 300 Hz, order 4)
##    and apply it zero-phase to 'raw_ap'. Store in 'ibl_lfp_band'.
## 2. Design a high-pass Butterworth filter (cutoff 300 Hz, order 4)
##    and apply it zero-phase to 'raw_ap'. Store in 'ibl_spike_band'.
ibl_lfp_band = ...
ibl_spike_band = ...
#####

fig, axes = plt.subplots(3, 1, figsize=(13, 9), sharex=True)
for ax, sig, label, color in zip(
    axes,
    [raw_ap, ibl_lfp_band, ibl_spike_band],
    ['Raw broadband (μV)', 'LFP band < 300 Hz (μV)', 'Spike band > 300 Hz (μV)'],
    ['C4', 'C0', 'C3']
):
    ax.plot(t_ap, sig, color=color, lw=0.6)
    ax.set_ylabel('Amplitude (μV)')
    ax.set_title(label)
axes[-1].set_xlabel('Time (s)')
plt.suptitle(f'#IBL Ch {BEST_CH} - LFP and Spike Band Separation',
             fontsize=13, fontweight='bold')
plt.tight_layout(); plt.show()
```



# Key Takeaways

- Filtering = convolution with a kernel  
= multiplication in frequency domain
- Sharp cutoffs create ringing (boxcar  $\rightarrow$  sinc)
- Causal filters delay; use zero-phase offline
- Band-pass a spike  $\rightarrow$  fake oscillation
- Apply these to real Neuropixels data